

A RUST TALK · 15 MINUTES

`Vec<Box<dyn Parser>>`

The compiler says **no**.

dyn-compatibility — what the language already gives you,
and how to cheat past the rest.

nixpulvis · a talk that ends in a crate

The code you wanted to write

THE PROBLEM

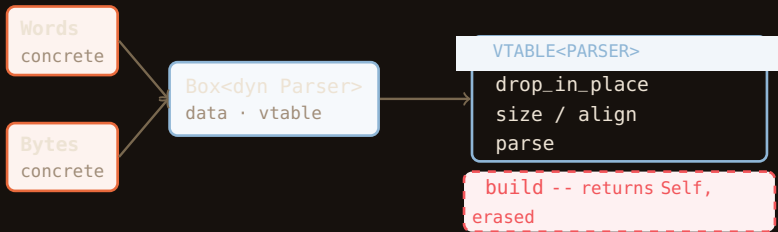
```
trait Parser {  
    fn parse(&self, input: &str) -> usize;  
    fn build() -> Self;  
}  
  
let parsers: Vec<Box<dyn Parser>> = vec![  
    Box::new(Words::build()),  
    Box::new(Bytes::build()),  
];
```

```
error[E0038]: the trait 'Parser'  
is not dyn compatible  
7 |   let parsers: Vec<Box<dyn Parser>>  
  |                               ^^^^^^not dyn  
  |                               compatible  
  
= note: for a trait to be dyn  
compatible it needs to allow  
building a vtable  
3 |   fn build() -> Self;  
  |       ^^^^^ ...because associated  
function 'build' has no 'self'  
parameter
```

real output -- rustc 1.88, verbatim (lightly wrapped)

There is no slot for `build`

THE PROBLEM



A vtable is a table of function pointers over an **erased** type. `build` returns `self` — the one thing erasure just deleted.

Everything that breaks `dyn`

THE PROBLEM

`fn build() -> Self;` returns the erased type

`fn crunch(self) -> u32;` consumes it by value

`fn write_to(&self, w: &mut impl Write);` different function per `w` — the vtable needs exactly one

`fn map<F>(self, f: F) -> Map<Self, F>;` generic: monomorphized, unbounded set

`trait Brush: Clone` supertrait `Clone` requires `Sized` — no method to blame

every one of these comes back in the next ten minutes

The escape hatch ships in the error message

WHAT THE LANGUAGE GIVES YOU

```
help: consider constraining 'build' so it does not apply to trait objects
3 |   fn build() -> Self where Self: Sized;
  |                   ++++++
```

where `Self: Sized` on a **method** removes that method from the vtable — not the trait from `dyn`. The trait stays dyn-compatible; the method is only callable on concrete types.

common misreading: people assume one bad method poisons the whole trait. It doesn't.

You already rely on this every day

WHAT THE LANGUAGE GIVES YOU

STD::ITER::ITERATOR

```
trait Iterator {  
    type Item;  
    fn next(&mut self)  
        -> Option<Self::Item>;  
  
    fn map<B, F>(self, f: F)  
        -> Map<Self, F>  
    where Self: Sized { ... }  
  
    // ~70 more adapters, each:  
    where Self: Sized
```

THE COMPILER WRITES THIS FOR FREE

```
impl Iterator  
    for dyn Iterator<Item=T> { ... }
```

The standard library's biggest trait is `dyn-compatible` because of this one clause — and `dyn Iterator` satisfies `Iterator` automatically, gated methods simply absent.

TAKEAWAY #1

**If you own the trait,
and its objects can live
without those methods —
you don't need a crate.**

where `Self: Sized`, done. The rest of this talk
is about when that isn't true.

Four walls you eventually hit

WHERE IT RUNS OUT

01

The method is dead, not dispatched

A `Self: Sized` method can never be called through the object. Fine for `Iterator::map` — not for a method your callers actually need on erased values.

03

The problem is a supertrait

`trait Brush: Clone` — there is no method to gate. The incompatibility lives on the trait itself.

02

You don't own the trait

`std::hash::Hash`, something from a dependency — you can't add a where-clause to code you can't edit.

04

`Box<dyn Foo>` is still not a `Foo`

`fn eat(x: impl Foo)` rejects your box. Hand-write the impl and the trap springs: the box **is** `Sized`, so every gated method is required again.

Generate the trait you'd have written by hand

THE CRATE • DYN_SHIM

YOU WRITE

```
#[dyn_shim(DynChannel)]
trait Channel {
    fn connect() -> Self;
    fn label(&self) -> String;
    fn deliver(&mut self, msg: &str);
    fn close(self) -> u32;
}
```

THE MACRO GENERATES (SIMPLIFIED)

```
trait DynChannel {
    fn label(&self) -> String;
    fn deliver(&mut self, msg: &str);
    fn close(self: Box<Self>) -> u32;
    // connect: skipped
}
impl<T: Channel> DynChannel for T {
    /*fwd */ }
}
```

A dyn-compatible twin plus a blanket impl: **every** `Channel` is already a `DynChannel` — SO `Vec<Box<dyn DynChannel>>` holds a mixed set of implementors.

Per method: forward, rewrite, or opt in

THE CRATE • METHOD SELECTION

<code>&self / &mut self / self: Box<Self></code>	forwarded through the vtable unchanged
<code>fn f(self)</code>	rewritten to <code>self: Box<Self></code> — automatic
<code>where Self: Sized</code>	already off the vtable — skipped; your gate is respected
<code>fn f(&self, w: &mut impl Write)</code>	#[dyn_shim(erase)] — lower the generic to a trait object
<code>fn f(self) -> Self</code>	#[dyn_shim(boxed)] — return <code>Box<dyn Shim></code> instead
<code>/* anything else */</code>	<code>stub = ... / panic / skipped</code>

Lower the generic to a trait object

REMEDICATION • ERASE

```
#[dyn_shim(DynLog)]  
trait Log {  
    #[dyn_shim(erase)]  
    fn write_to(&self, out: &mut impl Write);  
}
```

```
// on the shim:  
fn write_to(&self, out: &mut dyn Write);
```

Sound exactly when the bound gives you `impl Write` for `&mut dyn Write` — and `std` does, for **Hasher**, **Write**, **Read**, and friends.

A bound without it **fails at compile time**, at the forwarding call. Impossible erasures are rejected too:

```
error: #[dyn_shim(erase)] cannot erase 'T':  
it is not used behind a '&' or '&mut'  
argument
```

Builders keep building after erasure

REMEDICATION • BOXED

```
#[dyn_shim(DynBrush, reflexive = boxed)]
trait Brush {
  #[dyn_shim(boxed)]
  fn thicker(self) -> Self;
}

// on the shim:
fn thicker(self: Box<Self>)
  -> Box<dyn DynBrush>;
```

The concrete result is boxed and unsized back to the shim object, so a consuming, chaining `-> Self` builder still chains on erased values —

```
let b: Box<dyn DynBrush>
    = b.thicker().thicker();
```

— where `Self: Sized` would have simply deleted it from the object.

Clone and Hash aren't features — they're instances

THE GOOD PART

```
// std::hash::Hash - generic hasher  
fn hash<H: Hasher>(&self, s: &mut H);
```

ERASE

```
// the DynHash carrier  
fn hash(&self, s: &mut dyn Hasher);
```

```
// std::clone::Clone - returns Self  
fn clone(&self) -> Self;
```

BOXED

```
// the DynClone carrier  
fn clone_box(&self) -> Box<dyn DynClone>;
```

The shipped `DynClone` / `DynHash` carriers are generated by running synthetic signatures through the **same two transforms** user methods get. Two ecosystem crates, as one-liners of the general machinery.

Drop-in: retire `dyn-clone` and `dyn-hash`

IN PRACTICE

BEFORE -- TWO DEPS, TWO MACRO CALLS

```
# Cargo.toml
```

```
dyn-clone = "1"
```

```
dyn-hash  = "0.2"
```

```
use dyn_clone::{clone_trait_object, DynClone};
```

```
use dyn_hash::{hash_trait_object, DynHash};
```

```
trait MyTrait: DynClone + DynHash { }
```

```
clone_trait_object!(MyTrait);
```

```
hash_trait_object!(MyTrait);
```

AFTER -- ONE DEP, ONE ATTRIBUTE

```
# Cargo.toml
```

```
dyn_shim = { version = "0.2",  
             features = ["dyn_clone", "dyn_hash"] }
```

```
use dyn_shim::{dyn_shim_bind,
```

```
    DynClone, DynHash};
```

```
#[dyn_shim_bind(DynClone + DynHash)]
```

```
trait MyTrait: DynClone + DynHash { }
```

Supertraits stay put; the macro invocations become the attribute — same behavior, `Send`/`Sync` variants included.

Carriers mount capabilities onto objects

DYN_SHIM_BIND

```
// Foo is NOT dyn-compatible.  
// Widget can't have it as a supertrait...
```

```
#[dyn_shim_bind(DynFoo)]  
trait Widget: DynFoo {  
    fn draw(&self);  
}
```

```
// ...but now dyn Widget satisfies Foo,  
// forwarded through the inherited shim.
```

`DynClone` and `DynHash` aren't special: **any shim is a carrier**. Inherit it as a supertrait, bind the source trait back onto your objects — even from another crate.

Contract choice, made visible: a carrier **supertrait** means every implementor must qualify; a recognized bound on a shim merely **filters** which implementors become erasable.

Closing the loop back to the source trait

REFLEXIVE IMPLS

The language gives `dyn DynFoo: DynFoo` for free — but nothing ever makes `dyn DynFoo: Foo`. Two traits, one gap.

```
reflexive = bare
```

```
impl Foo for dyn DynFoo
```

Restores the impl the compiler would have written, had `Foo` been dyn-compatible. Gated methods left off the unsized surface: a compile error, not a runtime panic.

```
reflexive = boxed
```

```
impl Foo for Box<dyn DynFoo>
```

The real feature. `Self` is sized, so by-value `self` and boxed builders genuinely work — and the remediation ladder replaces stubs you'd hand-write. Your box now passes into `fn eat(x: impl Foo).`

Under everything: two primitives

IMPLEMENTATION

`erase_generic(sig)`

generic param behind `&/&mut` → trait object — consumed by `#[dyn_shim(erase)]` and the **Hash carrier**

`build_boxed_method(sig, call)`

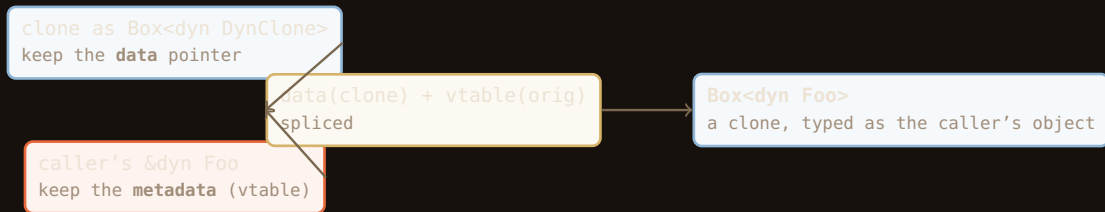
box a `Self`-typed result, unsize to `Box<dyn Shim + markers>` — consumed by `#[dyn_shim(boxed)]` and the **Clone carrier**

The user-facing attributes and the shipped carriers are the **same code paths**. The macro is smaller than its feature list — the features are points in a two-transform space.

the Hash carrier is literally a synthetic `fn(&self, &mut impl Hasher) run through erase_generic`

The one unsafe block: fat-pointer surgery

IMPLEMENTATION



The carrier can only produce `Box<dyn DynClone>`; the caller wants `Box<dyn Foo>`. Same allocation, same concrete type — only the metadata differs. So splice the fresh data pointer into the original's metadata, with an `assert_eq!` pinning the layout assumption.

For a proc macro, the errors are the API

IMPLEMENTATION

```
$ cat tests/ui/erase_by_value.stderr
```

```
error: #[dyn_shim(erase)] cannot erase 'T': it is not used  
behind a '&' or '&mut' argument (only such uses lower to a  
trait object)
```

```
-> tests/ui/erase_by_value.rs:10:13  
  |  
10 |     fn sink<T: std::fmt::Debug>(&self, value: T);  
  |           ^
```

41 trybuild UI tests pin the exact wording of every rejection — each

impossible trait, method, and erasure fails with a diagnostic that

19 / 21 says why, at the right span. Change the message, break the build. `dyn_shim`

The decision ladder

RECAP

`where Self: Sized`

own the trait, objects can lose the methods — the language has you; no dependency

`#[dyn_shim(DynFoo)]`

methods must dispatch, or you need
`Box<dyn>: Foo` — shim + erase / boxed / reflexive

`#[dyn_shim_foreign(path::Foo)]`

the trait isn't yours — restate it, the macro forwards and blanket-impls

`#[dyn_shim_bind(DynClone + ..)]`

trait is fine, objects need Clone / Hash / Foo — mount capabilities in place

Rung one is the language; the crate is rungs two through four.

FIN

You might not need
dyn_shim.

But now you know what it's for.

github.com/nixpulvis/dyn_shim · crates.io/crates/dyn_shim · [questions?](#)